

EN UNA FRASE Sabes sustituir el repositorio en memoria por una base de datos real: conviertes `Libro` en entidad JPA, obtienes el CRUD extendiendo `JpaRepository` y trabajas con H2 en desarrollo dejando PostgreSQL preparado para producción.

LO ESENCIAL

JPA / Hibernate / Spring Data JPA	Especificación (anotaciones e interfaces) / implementación que ejecuta el SQL / capa que genera los repositorios
spring-boot-starter-data-jpa	Starter con Hibernate, Spring Data JPA y el pool HikariCP
com.h2database:h2 (scope runtime)	Base de datos H2 en memoria para desarrollo y tests
@Entity + @Table(name = "Libro")	Marca la clase como entidad y fija el nombre de la tabla
@Id + @GeneratedValue(strategy = GenerationType.IDENTITY)	Clave primaria con autoincremento delegado en la base de datos
@Column(length = 200)	Longitud de la columna <code>VARCHAR</code> , a juego con el <code>@Size</code> del campo
Requisitos de una entidad	Constructor vacío, getters y setters, <code>id</code> de tipo <code>Long</code> (no <code>long</code>) y clase no <code>final</code>
JpaRepository<Libro, Long>	Interfaz con el CRUD completo: <code>findAll</code> , <code>findById</code> , <code>save</code> , <code>deleteById</code> , <code>existsById</code> , <code>count</code>
save(libro)	Con <code>id</code> nulo hace INSERT; con <code>id</code> , UPDATE
Query Derivation	Consultas a partir del nombre del método: <code>findByAutor</code> , <code>findByAnioGreaterThan</code> , <code>findByTituloContainingIgnoreCase</code>
spring.jpa.hibernate.ddl-auto	<code>create-drop</code> en desarrollo; <code>update</code> en producción hasta que llegue Flyway, y entonces <code>validate</code>
data.sql	Inserta los datos iniciales al arrancar
spring.jpa.defer-datasource-initialization=true	Hace que <code>data.sql</code> se ejecute después de que Hibernate cree las tablas
spring.h2.console.enabled=true	Activa la consola web de H2 en <code>/h2-console</code> (solo en el perfil <code>dev</code>)

ERRORES TÍPICOS

1 InstantiationException: No default constructor for entity

✓ Hibernate crea las instancias por reflexión; añade siempre `public Libro() {}`.

2 Table "LIBRO" not found al ejecutar data.sql

✓ el script se lanza antes de que Hibernate cree las tablas; añade `spring.jpa.defer-datasource-initialization=true`.

3 Los datos desaparecen tras reiniciar en producción

✓ `create-drop` es solo para desarrollo; en `application-prod.properties` usa `spring.jpa.hibernate.ddl-auto=update` (y `validate` cuando Flyway gestione el esquema).

4 save() hace UPDATE en lugar de INSERT con un libro nuevo

✓ el `id` es `long` primitivo y vale 0, nunca `null`; decláralo como `Long`.

5 La consola H2 aparece vacía o sin la tabla LIBRO

✓ te has conectado a la URL por defecto del formulario (`jdbc:h2:~/test`); escribe exactamente `jdbc:h2:mem:biblioteca`.

EN EL PROYECTO BIBLIOTECA

`Libro` pasa a ser entidad JPA, se crea `LibroJpaRepository` y `BibliotecaService` lo usa en lugar de `RepositorioLibros`, que se elimina junto con `CargadorDatosPrueba`. Los tres libros iniciales se cargan ahora desde `data.sql`. `application-dev.properties` configura H2 con `create-drop` y la consola web; `application-prod.properties` queda preparado para PostgreSQL con `ddl-auto=update`, y el `pom.xml` incorpora `spring-boot-starter-data-jpa`, `h2` y el driver `postgresql`. El controlador y sus tests no cambian.



Spring Data JPA y base de datos

■ CÓDIGO MÍNIMO Y PETICIONES

La entidad `Libro` (recortada):

```
@Entity
@Table(name = "libro")
public class Libro {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @NotBlank(message = "El título es obligatorio")
    @Size(min = 1, max = 200, message = "El título no puede superar los 200 caracteres")
    @Column(length = 200)
    private String titulo;

    public Libro() {
    }
}
```

JAVA

El repositorio: basta con declarar la interfaz.

```
package com.javaesencial.biblioteca.repositorio;

import com.javaesencial.biblioteca.dominio.Libro;
import org.springframework.data.jpa.repository.JpaRepository;

public interface LibroJpaRepository extends JpaRepository<Libro, Long> {
}
```

JAVA

H2 y JPA en `application-dev.properties`:

```
spring.datasource.url=jdbc:h2:mem:biblioteca
spring.datasource.driver-class-name=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.h2.console.enabled=true

spring.jpa.hibernate.ddl-auto=create-drop
spring.jpa.show-sql=true
spring.jpa.defer-datasource-initialization=true
```

PROPERTIES

```
-- Consola H2: http://localhost:8080/h2-console
-- JDBC URL: jdbc:h2:mem:biblioteca · User Name: sa · Password: (vacío)

SELECT * FROM LIBRO;
SELECT * FROM LIBRO WHERE ANIO > 2000;
SHOW COLUMNS FROM LIBRO;
```

SQL